NexGen CI/CD Pipeline — Setup, Operation & Replication Guide

Repository: `nexgen-it/nexgen.e-meril.in` · **GitLab:** `git.e-meril.in` **Last verified:** 21 Aug 2026 · 151 tests passing, full pipeline green

1. What the pipeline does

One command starts everything:

```
git push origin <your-branch>:staging
```

From there:

```
   push to staging
        |
   [check]    syntax-check · code-quality · sonarqube
        |
   [smoke]    build image -> boot full stack -> migrate -> collectstatic
              -> HTTP check -> django check -> migration check
              -> 151 tests -> frontend integrity
        |
   [build]    build-image (tag = commit SHA) -> push to registry
        |
   [promote]  fast-forward main to staging  (refuses anything else)
        |
   [deploy]   deploy-production   <-- MANUAL BUTTON
              pull tag on prod -> restart -> health check
              -> roll back automatically if unhealthy
```

Every stage gates the next. There is deliberately **no** `needs:` **DAG**, so a failure anywhere stops everything downstream.

2. Architecture

Host	Role	Details
`OLMWST0027`	Dev box + **GitLab runner**	Shell executor, jobs run as `gitlab-runner` (uid 997), rootless podman
`mldcs033` (`10.120.130.31`)	**Production**	SSH on port **7722**, user `vramani` , containers under **rootful** podman, app dir `/home/vramani/nexgen.e-meril.in`
`git.e-meril.in`	GitLab server	Also the container registry at `registry-gitlab.e-meril.in`

Production hosts roughly 28 containers (Jenkins, Metabase, OpenProject, several databases) — which is why the deploy's root access is tightly scoped.

3. Files involved

File	Purpose
`.gitlab-ci.yml`	The pipeline definition — all stages and jobs
`docker-compose.ci.yml`	Isolated throwaway stack used by the smoke test
`scripts/ci-smoke.sh`	Builds, boots and verifies the stack; the main gate
`scripts/ci-entrypoint.sh`	Container startup for the CI stack
`scripts/ci-quality.sh`	black · isort · flake8 · bandit, in a container
`scripts/ci_frontend_check.py`	Renders pages, verifies every static asset resolves
`scripts/deploy-production.sh`	Runs on the runner; drives the deploy over SSH
`scripts/prod/nexgen-deploy`	**Installed on prod** as the only root command CI may run
`sonar-project.properties`	SonarQube project key and exclusions
`docker-compose.registry.yml`	Production runtime (pulls image by tag)

4. Stage-by-stage

4.1 `check` — runs on every branch

- **`syntax-check`** — `flake8 --select=E9,F63,F7,F82`. Fatal errors only: syntax errors, undefined names, invalid `print` / `assert`. No style rules, so it never blocks on formatting. Runs in a throwaway container so the runner host's Python is untouched.
- **`code-quality`** (allow_failure) — black, isort, flake8 (full), bandit. Every tool runs even if an earlier one fails, so a formatting nit cannot hide a security finding.
- **`sonarqube`** (allow_failure) — runs **only if `SONAR_HOST_URL` is set**, so the pipeline stays green for anyone without a Sonar server.

4.2 `smoke` — staging only, the real gate

Brings up `docker-compose.ci.yml`: a throwaway Postgres (data on tmpfs) plus the app built from this commit. Then asserts, in order:

1. **Container appeared** within 30s — `podman-compose` exits 0 even when a service fails to start, so this is checked explicitly.
2. **HTTP 2xx/3xx** on `/` with `X-Forwarded-Proto: https`.
3. **`manage.py check`** passes inside the container.
4. **No missing migrations** — `makemigrations --check --dry-run`.
5. **151 tests pass** — `manage.py test` with `DJANGO_DEBUG=True`.
6. **Frontend integrity** — pages render and every referenced static asset was actually collected.

Always tears down via a trap on `EXIT` / `INT` / `TERM`, including when GitLab cancels the job. `resource_group: smoke` serialises it so two pipelines cannot both bind host port 8021.

4.3 `build` — publish the verified artifact

Builds the commit smoke just proved, tags it with the **full commit SHA**, and pushes to the registry using `CI_REGISTRY_USER` / `CI_REGISTRY_PASSWORD` (the job token — no secret of ours involved).

4.4 `promote` — fast-forward only

Clones with `CI_JOB_TOKEN` (always has read access), then:

- no-op if `main` already equals this commit;
- **refuses unless it is a clean fast-forward** (`git merge-base --is-ancestor`), so a direct commit on `main` can never be silently lost;
- pushes with `PROMOTE_TOKEN`.

4.5 `deploy` — manual, with rollback

Runs `scripts/deploy-production.sh` on the runner, which drives `/usr/local/bin/nexgen-deploy` on prod over SSH:

1. Validate the SSH key locally (armor lines present).
2. Preflight — prove the wrapper is reachable before touching anything.
3. Record the currently running `APP_TAG` as the rollback target.
4. **Pull first** — a bad image costs nothing, production keeps serving.
5. Restart with the new tag.
6. Health check from on prod, up to 180s.
7. **Roll back to the previous tag** if it never becomes healthy.
8. Verify the public URL (warn-only — see §6).

---

5. Setting it up from scratch

5.1 Prerequisites

On the runner host: `podman`, `podman-compose`, `git`, `curl`. On production: `podman`, `podman-compose`, `curl`, and the app's compose file.

5.2 Install and register the runner

```
curl -L "https://packages.gitlab.com/install/repositories/runner/gitlab-runner/script.deb.sh" |
sudo bash
sudo apt-get install -y gitlab-runner
```

In GitLab: **Settings -> CI/CD -> Runners -> Create project runner**. Tick **"Run untagged jobs"** (the jobs carry no `tags:`). Then:

```
sudo gitlab-runner register --non-interactive \
  --url https://git.e-meril.in \
  --token glrt-XXXXXXXX \
  --executor shell \
  --description "nexgen podman runner"
```

Use the **internal** hostname if you have one — it keeps registration and every `git fetch` off the public network.

5.3 The subuid gotcha (WILL bite you)

Shell-executor jobs run as the `gitlab-runner` user. The package creates that account **without** a `subuid`/`subgid` range, so rootless podman fails with "no subuid ranges found for user".

```
# pick a free range; check /etc/subuid for what is taken
sudo usermod --add-subuids 427680-493215 --add-subgids 427680-493215 gitlab-runner
sudo loginctl enable-linger gitlab-runner
sudo systemctl restart gitlab-runner
```

Verify — this is the go/no-go test:

```
sudo -u gitlab-runner -H bash -lc 'cd /tmp; podman run --rm docker.io/library/alpine echo ok'
```

Also raise the runner's **Maximum job timeout** to 3600s: a cold build with no cached pip layer can exceed the 30-minute default.

5.4 Production wrapper + narrow sudoers

Install `scripts/prod/nexgen-deploy` on production as root:

```
sudo install -o root -g root -m 755 nexgen-deploy /usr/local/bin/nexgen-deploy
```

Grant exactly one command — nothing wider:

```
cat > /etc/sudoers.d/nexgen-deploy <<'EOF'
vramani ALL=(root) NOPASSWD: /usr/local/bin/nexgen-deploy
EOF
chmod 440 /etc/sudoers.d/nexgen-deploy
visudo -c
grep -rn "vramani" /etc/sudoers /etc/sudoers.d/    # must show exactly ONE line
```

Why not `NOPASSWD: /bin/sh`? Because that is unrestricted root over every container on the host, handed to whoever holds the CI SSH key — and that key lives in a CI variable editable by anyone who can edit `.gitlab-ci.yml`. The wrapper fixes every path, image and container name internally; the caller supplies only an image tag, which is validated against `[A-Za-z0-9._-]` and rejected if it looks like a flag or contains `..`.

5.5 Registry access on production

Create a **Deploy token** (Settings -> Repository -> Deploy tokens) with scope `read_registry` **only**, then on prod as root:

```
podman login registry-gitlab.e-meril.in
```

Credentials persist in root's `auth.json`, so **no registry secret ever passes through CI**.

5.6 SSH key for the deploy

```
ssh-keygen -t ed25519 -f /root/.ssh/nexgen_deploy -N "" -C "gitlab-ci-deploy"
ssh-copy-id -i /root/.ssh/nexgen_deploy.pub -p 7722 vramani@10.120.130.31
```

Verify with **no password prompt** — the job uses `BatchMode=yes` and cannot fall back to a password:

```
ssh -i /root/.ssh/nexgen_deploy -p 7722 vramani@10.120.130.31 \
  'sudo -n /usr/local/bin/nexgen-deploy health'
```

5.7 CI/CD variables

Settings -> CI/CD -> Variables. Leave **"Protected" unticked on all of them** unless `staging` is itself a protected branch.

Key	Type	Example	Notes
`PROMOTE_TOKEN`	Variable	`glpat-…`	Project Access Token, role **Maintainer**, scopes `read_repository` + `write_repository`
`PROD_HOST`	Variable	`10.120.130.31`	
`PROD_USER`	Variable	`vramani`	
`PROD_PORT`	Variable	`7722`	
`PROD_USE_SUDO`	Variable	`1`	Set when prod containers run as root
`PROD_URL`	Variable	`https://nexgen.e-meril.in/`	Environment link + warn-only check
`PROD_SSH_KEY`	**File**	contents of the private key	**Must be type File.** Include the `-----BEGIN` / `-----END` lines
`SONAR_HOST_URL`	Variable	(optional)	Omit to skip the Sonar job
`SONAR_TOKEN`	Variable	(optional)	Masked

5.8 Protected branch

Settings -> Repository -> Protected branches -> `main` -> Allowed to push and merge must include **Maintainers**, or the promote step is rejected.

5.9 Verification checklist

Work through these in order; each is cheap and isolates one layer:

1. `sudo -u gitlab-runner -H bash -lc 'cd /tmp; podman run --rm docker.io/library/alpine echo ok'`
2. `./scripts/ci-smoke.sh` (locally — the same script CI runs)
3. `ssh -i <key> -p 7722 <user>@<host> 'sudo -n /usr/local/bin/nexgen-deploy current-tag'`
4. `... nexgen-deploy health` -> expect **200**
5. Push to `staging` and watch the pipeline
6. Click **deploy** and watch `sudo podman ps` on prod

---

6. Design decisions and why they matter

Build once, deploy that artifact. The image is built and tagged with the commit SHA in CI, and production pulls that tag. It never rebuilds. So what ships is byte-for-byte what the tests ran against. Rollback is re-pulling an older tag — seconds, no rebuild.

Promotion is fast-forward only. `promote-to-main` refuses anything that is not a clean fast-forward. If someone commits straight to `main`, the pipeline stops rather than discarding their work.

Production deploys are manual. CI to staging is automatic; production is a deliberate click with a full audit trail in GitLab's Deployments view. This is the industry norm and it costs nothing.

Pull before restart. A missing or corrupt image fails the pull, and production keeps serving the old one. Only after a successful pull is anything restarted.

Rollback is tied to the app's own health, not the public URL. The gate is `localhost:8020` on prod — that tests the container. The public URL is checked separately and **warn-only**: if the app is healthy but the proxy is broken, rolling back would revert a good deploy and fix nothing.

Root access on prod is one fixed command. See §5.4.

The CI stack cannot touch production data. `docker-compose.ci.yml` has no `env_file:`, so it never reads `.env` — whose `DB_HOST` points at the production database. It also uses its own port, its own network and its own throwaway Postgres.

Every check must be able to fail. Each gate was verified in both directions — passing on good input and failing on bad. Two checks initially reported success while verifying nothing (see §7).

---

7. Gotchas discovered the hard way

These cost real debugging time. If you replicate this elsewhere, read them first.

1. `curl` prints `000` and exits non-zero. So `code=$(curl -w '%{http_code}' … || echo 000)` yields `000000`, which passes an `-lt 500` test as integer zero. A dead app was reported healthy. Capture the output, then normalise separately.

2. `podman-compose ps -q <service>` is not supported (podman-compose 1.0.6 rejects the service argument). Query the runtime by label instead: `com.docker.compose.project` + `com.docker.compose.service`.

3. `podman-compose up -d` exits 0 even when a service fails to start. Check explicitly that the container appeared, or you will wait out the entire boot timeout for no reason.

4. `SECURE_SSL_REDIRECT` breaks every HTTP check when `DEBUG=False`. A plain probe gets `301` from middleware before any view runs. Send `X-Forwarded-Proto: https` (settings already honour `SECURE_PROXY_SSL_HEADER`). The same trap fails **16 tests** — run the suite with `DJANGO_DEBUG=True`.

5. Health gates must require 2xx/3xx, not `< 500`. Django answers **400 DisallowedHost** when the `Host` header is not in `ALLOWED_HOSTS`, and a `<500` check happily passes that. Send an accepted `Host` and tighten the threshold.

6. Short image names may not resolve. `postgres:16-alpine` failed on a clean CI user: this host defines no unqualified-search registries, and `/etc/containers/registries.conf.d` aliases `python` and `alpine` but **not** `postgres`. It only appeared to work locally because the image was already cached. **Fully qualify every image name** you control.

7. Podman storage is per-user. If production containers run as root and the deploy connects as an unprivileged user, the deploy will pull the image, start a second container in that user's storage, never touch the live one, and report success. A green pipeline that deployed nothing. Check with `podman inspect <container>` as both users.

8. Pasting a private key into a web form loses the newline after `-----BEGIN`. OpenSSL then reports only "error in libcrypto", which names nothing. `scripts/deploy-production.sh` repairs this automatically and prints the key's fingerprint on failure.

9. `write_registry` is not `write_repository`. Adjacent in the scope list, nearly identical names, completely different grants. The wrong one produces "You are not allowed to upload code".

10. A "Protected" CI variable is not injected on an unprotected branch. It silently does not exist, and your job reports the variable as unset even though you can see it in the UI.

11. Cloning needs read access. Use `CI_JOB_TOKEN` for the clone — every job gets one with read access to its own project — and reserve your own token for the push. That removes a whole class of scope misconfiguration.

12. A stray `__init__.py` at the repository root breaks test discovery. It makes the project directory a Python package, so unittest imports every app as `app.<name>` and Django cannot resolve app labels. **128 of 151 tests were never being collected.**

13. A manual script named `test_*.py` gets imported by the test runner. If it executes anything at module scope — an ORM query, for instance — the whole suite errors. Name scripts something else.

14. A named volume over `STATIC_ROOT` outlives deploys. `collectstatic` without `--clear` only ever adds and overwrites, and it skips files whose target is not older than the source. Result: Python updates with the image while the frontend serves old assets. Use `collectstatic --noinput --clear`.

15. Nothing stops a downgrade deploy. Fast-forward checks compare `main` to `staging`; they say nothing about whether the commit being deployed is older than what is already running. Deploying a commit that predated a bug fix reintroduced the bug in production. **This remains unguarded — see §10.**

---

8. Day-to-day usage

```
# work on your branch, then ship it
git add -A && git commit -m "..."
git push origin <your-branch>:staging
```

Watch the pipeline, then click **deploy-production** on the `main` pipeline when you are ready. Roughly 3-5 minutes to the deploy button; ~60s for the deploy.

Do not commit directly to `staging` or `main`. Everything should flow from your working branch. If `main` diverges, promotion refuses:

```
main has diverged from staging — not a fast-forward.
```

Recover with `git checkout <branch> && git merge origin/main` and push again.

---

9. Rollback and incident response

Automatic: if the app does not become healthy within 180s, the deploy restores the previous `APP_TAG` and fails the job.

Manual, on production:

```
sudo /usr/local/bin/nexgen-deploy current-tag        # what is live
sudo /usr/local/bin/nexgen-deploy restart <old-sha>  # roll back
sudo /usr/local/bin/nexgen-deploy health             # expect 200
sudo /usr/local/bin/nexgen-deploy logs               # last 60 lines
```

Any previously deployed SHA works as the argument. GitLab's **Deployments** page also offers a re-deploy button per environment.

---

10. Replicating this on another repository

1. Copy `scripts/` (`ci-smoke.sh`, `ci-entrypoint.sh`, `ci-quality.sh`, `ci_frontend_check.py`, `deploy-production.sh`, `prod/nexgen-deploy`), `docker-compose.ci.yml` and `.gitlab-ci.yml`.
2. Edit `docker-compose.ci.yml`: service names, ports, environment. Keep it free of `env_file:` so it can never reach a real database.
3. Edit `scripts/prod/nexgen-deploy`: `APP_DIR`, `IMAGE`, `CONTAINER`, `HEALTH_URL`, `HEALTH_HOST` — all fixed inside the wrapper by design.
4. Edit `scripts/ci_frontend_check.py`: the `DEFAULT_PAGES` list.

5. Adjust `sonar-project.properties`: project key plus exclusions for whatever vendored or generated code that repo has.
6. Work through §5 on the new hosts.
7. Run §5.9's checklist before trusting any of it.

Known gap worth closing first: the downgrade guard (§7, item 15). Production records `APP_TAG` as a commit SHA, so the deploy job can read it and refuse when the incoming commit is an ancestor of the running one, with an override variable for deliberate rollbacks.

---

11. Current status

- 151 tests passing (was 23 discovered, 16 failing)
- Full pipeline verified green end to end
- `code-quality` reports 217 black / 77 isort / 1024 flake8 / 22 bandit findings — deliberately `allow_failure` until that backlog is worked down
- Frontend check verified in both directions: passes clean, and catches a deleted CSS file

---

12. Regenerating this PDF

`CICD_PIPELINE_GUIDE.md` is the source of truth. After editing it, rebuild the PDF with the same containerised toolchain the pipeline uses elsewhere — nothing needs installing on the host:

```
podman run --rm -v "$PWD":/w -w /w docker.io/library/python:3.13-slim sh -c '
  apt-get update -qq
  apt-get install -y -qq --no-install-recommends \
      libpango-1.0-0 libpangoft2-1.0-0 libharfbuzz0b fonts-dejavu-core
  pip install --quiet weasyprint markdown
  python docs/build_pdf.py docs/CICD_PIPELINE_GUIDE.md docs/CICD_PIPELINE_GUIDE.pdf'
```

`docs/build_pdf.py` holds the Markdown-to-PDF conversion and the print stylesheet (A4, page numbers, table and code-block styling). It works for any Markdown file in this repo, not just this one.